

Advanced SVMs

Hrachya Asatryan

Problem Formulation

What do we have?

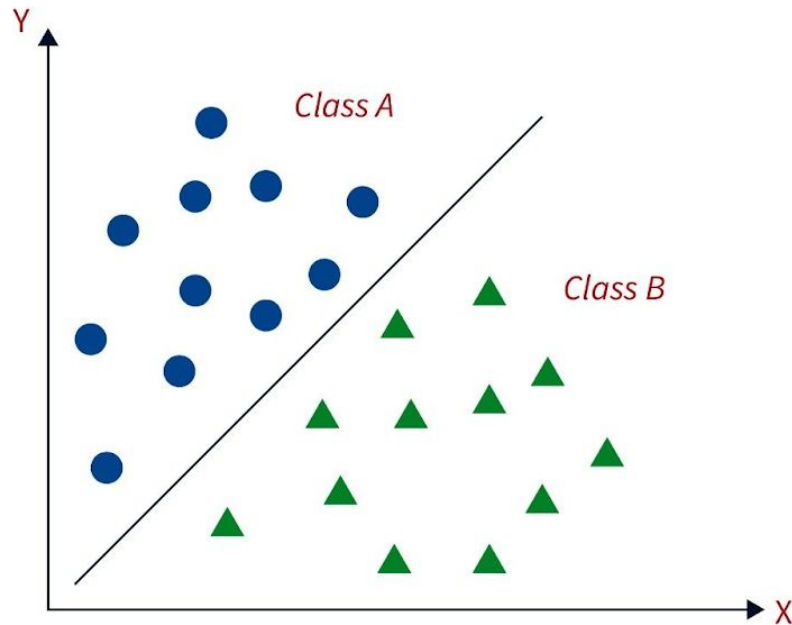
A training data set comprising $\mathbf{N}$ observations $\{\mathbf{x}_n\}$, where $n = 1, \dots, N$, together with corresponding target values $t_1, \dots, t_N$, $t_n \in \{-1, 1\}$

What is our goal?

Take an input vector $\mathbf{x}$ and to assign it to one of K classes, while searching the solutions in a sparse space.

How do we do that?

We try to maximize the distance between the closest points to the decision boundary.



Maximum Margin Classifiers

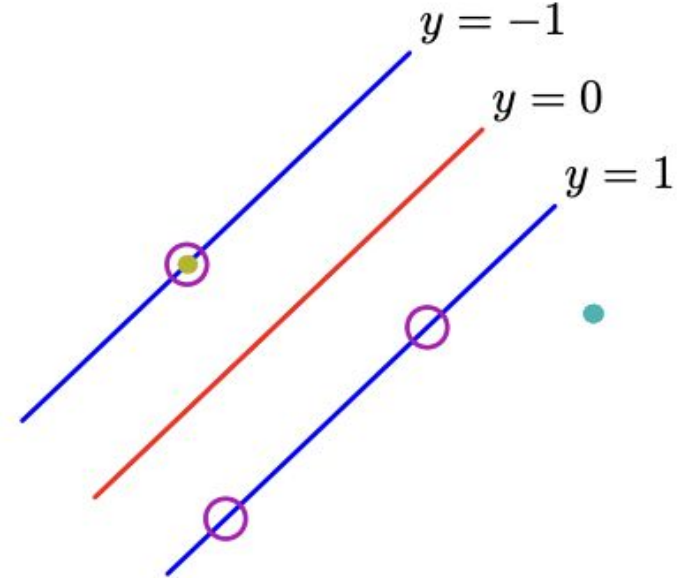
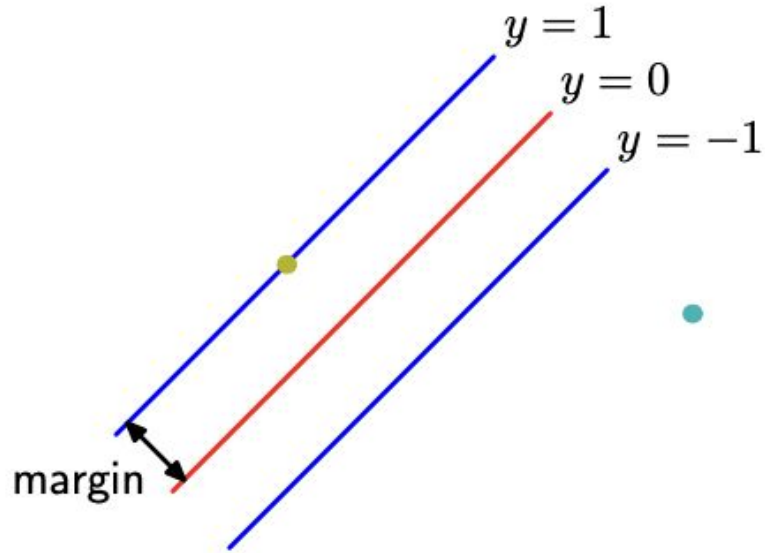
Problem Formulation:

We return to our original problem of two class classification, given $y(\mathbf{x}) = \mathbf{w}^T \varphi(\mathbf{x}) + b$, where $\varphi(\mathbf{x})$ denotes a fixed feature-space transformation, and we have made the bias parameter b explicit.

The training data set comprises N input vectors $x_1, \dots, x_N$ with corresponding target values $t_1, \dots, t_N$ where $t_n \in \{-1, 1\}$, and new data points x are classified according to the sign of $y(\mathbf{x})$. We shall assume for the moment that the training data set is linearly separable in feature space, so that by definition there exists at least one choice of the parameters $\mathbf{w}$ and b such that $y(x_n) > 0$ for points having $t_n = +1$ and $y(x_n) < 0$ for points having $t_n = -1$, thus $t_n y(x_n) > 0$ for all training data points.

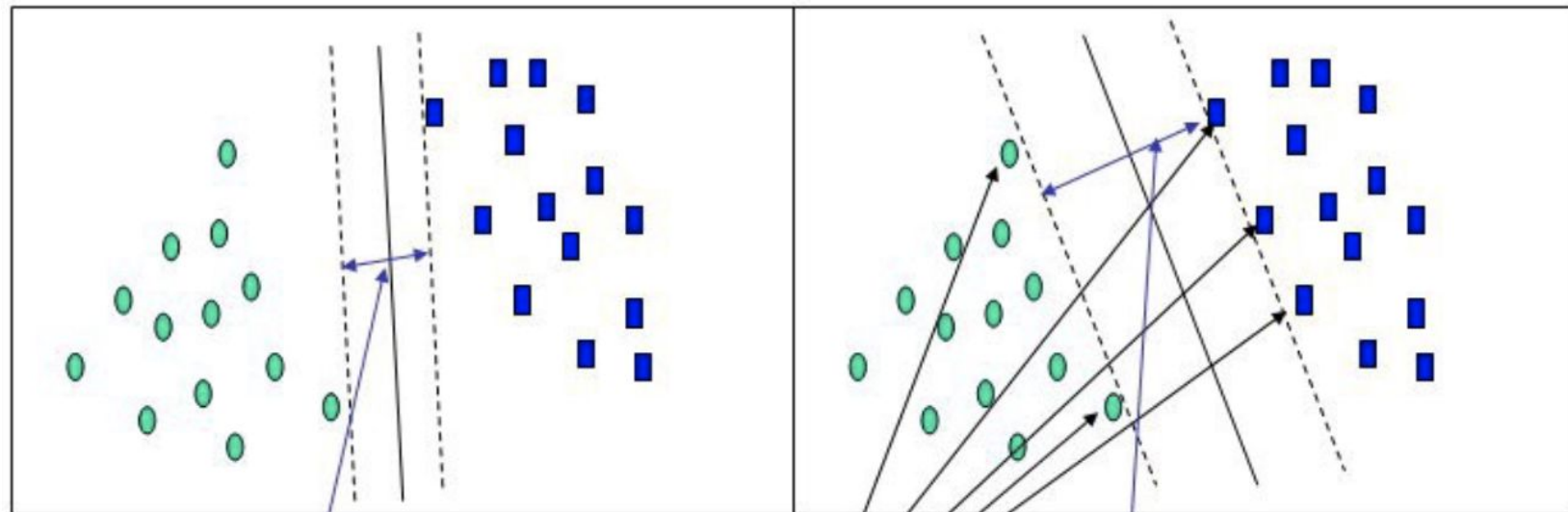
If there are multiple solutions all of which classify the training data set exactly, then we should try to find the one that will give the smallest generalization error. The **support vector machine** approaches this problem through the concept of the **margin**, which is defined to be the smallest distance between the **decision boundary** and any of the samples.

Maximum Margin Classifiers



The margin is defined as the perpendicular distance between the decision boundary and the closest of the data points, as shown on the left figure. Maximizing the margin leads to a particular choice of decision boundary, as shown on the right. The location of this boundary is determined by a subset of the data points, known as **support vectors**, which are indicated by the circles.

Margins



Small Margin

Large Margin

Support Vectors

Maximum Margin Classifiers

Main Algorithm:

The perpendicular distance of a point $\mathbf{x}$ from a hyperplane defined by $y(\mathbf{x})=0$ is given by $|y(\mathbf{x})|/\|\mathbf{w}\|$. Furthermore, we are only interested in solutions for which all data points are correctly classified, so that $t_n y(\mathbf{x}_n) > 0$ for all n . Thus the distance of a point $\mathbf{x}_n$ to the decision surface is given by:

$$\frac{t_n y(\mathbf{x}_n)}{\|\mathbf{w}\|} = \frac{t_n (\mathbf{w}^T \phi(\mathbf{x}_n) + b)}{\|\mathbf{w}\|}$$

The margin is given by the perpendicular distance to the closest point $\mathbf{x}_n$ from the data set, and we wish to optimize the parameters $\mathbf{w}$ and b in order to maximize this distance. Thus the maximum margin solution is found by solving:

$$\arg \max_{\mathbf{w}, b} \left\{ \frac{1}{\|\mathbf{w}\|} \min_n [t_n (\mathbf{w}^T \phi(\mathbf{x}_n) + b)] \right\}$$

where we have taken the factor $1/\mathbf{w}$ outside the optimization over n because $\mathbf{w}$ does not depend on n

Maximum Margin Classifiers

Main Algorithm:

Solving this optimization problem directly would be very complex, and so we shall convert it into an equivalent problem that is much easier to solve. To do this we note that if we make the rescaling $\mathbf{w} \rightarrow \kappa \mathbf{w}$ and $b \rightarrow \kappa b$, then the distance from any point $\mathbf{x}_n$ to the decision surface, given by $t_n y(\mathbf{x}_n) / \|\mathbf{w}\|$, is unchanged. We can use this freedom to set

$$t_n (\mathbf{w}^T \phi(\mathbf{x}_n) + b) = 1$$

for the point that is closest to the surface. In this case, all data points will satisfy the constraints

$$t_n (\mathbf{w}^T \phi(\mathbf{x}_n) + b) \geq 1, \quad n = 1, \dots, N.$$

This is known as the canonical representation of the decision hyperplane. In the case of data points for which the equality holds, the constraints are said to be **active**, whereas for the remainder they are said to be **inactive**. By definition, there will always be **at least one active constraint**, because there will always be a **closest point**, and once the margin has been maximized there will be at least two active constraints.

Maximum Margin Classifiers

Main Algorithm:

The optimization problem then simply requires that we maximize $\|\mathbf{w}\|^{-1}$, which is equivalent to minimizing $\|\mathbf{w}\|^2$, and so we have to solve the optimization problem, subject to the constraints given above:

$$\arg \min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

It appears that the bias parameter b has disappeared from the optimization. However, it is determined implicitly via the constraints, because these require that changes to $\mathbf{w}$ be compensated by changes to b .

In order to solve this constrained optimization problem, we introduce Lagrange multipliers $a_n \geq 0$, with one multiplier a_n for each of the constraints, giving the Lagrangian function the form:

$$L(\mathbf{w}, b, \mathbf{a}) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{n=1}^N a_n \{t_n(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n) + b) - 1\}$$

Maximum Margin Classifiers

Main Algorithm:

where $\mathbf{a} = (a_1, \dots, a_N)^T$. Note the minus sign in front of the Lagrange multiplier term, because we are minimizing with respect to $\mathbf{w}$ and b , and maximizing with respect to $\mathbf{a}$. Setting the derivatives of $L(\mathbf{w}, b, \mathbf{a})$ with respect to $\mathbf{w}$ and b equal to zero, we obtain the following two conditions:

$$\mathbf{w} = \sum_{n=1}^N a_n t_n \phi(\mathbf{x}_n) \quad 0 = \sum_{n=1}^N a_n t_n.$$

Eliminating $\mathbf{w}$ and b from $L(\mathbf{w}, b, \mathbf{a})$ using these conditions then gives the **dual representation** of the maximum margin problem in which we maximize

$$\tilde{L}(\mathbf{a}) = \sum_{n=1}^N a_n - \frac{1}{2} \sum_{n=1}^N \sum_{m=1}^N a_n a_m t_n t_m k(\mathbf{x}_n, \mathbf{x}_m)$$

Maximum Margin Classifiers

Main Algorithm:

The **dual representation** is subject to constraints such as:

$$a_n \geq 0, \quad n = 1, \dots, N, \quad \sum_{n=1}^N a_n t_n = 0.$$

Here the kernel function is defined by $k(\mathbf{x}, \mathbf{x}') = \boldsymbol{\varphi}(\mathbf{x})^\top \boldsymbol{\varphi}(\mathbf{x}')$. The solution to a quadratic programming problem in M variables in general has computational complexity that is $O(M^3)$. In going to the dual formulation we have turned the original optimization problem into the dual problem, which has N variables. For a fixed set of basis functions whose number M is smaller than the number N of data points, the move to the dual problem appears disadvantageous. However, it allows the model to be reformulated using kernels, and so the **maximum margin classifier** can be applied efficiently to feature spaces whose **dimensionality exceeds the number of data points**, including **infinite feature spaces**. The kernel formulation also makes clear the role of the constraint that the kernel function $k(\mathbf{x}, \mathbf{x}')$ be positive definite, because this ensures that the Lagrangian function is bounded below, giving rise to a well defined optimization problem.

Maximum Margin Classifiers

Main Algorithm:

In order to classify new data points using the trained model, we evaluate the sign of $y(\mathbf{x})$. This can be expressed in terms of the parameters $\{a_n\}$ and the kernel function by substituting for $\mathbf{w}$, to give

$$y(\mathbf{x}) = \sum_{n=1}^N a_n t_n k(\mathbf{x}, \mathbf{x}_n) + b.$$

Here we have 3 constraints:

$$\begin{aligned} a_n &\geq 0 \\ t_n y(\mathbf{x}_n) - 1 &\geq 0 \\ a_n \{t_n y(\mathbf{x}_n) - 1\} &= 0. \end{aligned}$$

Thus for every data point, either $a_n = 0$ or $t_n y(\mathbf{x}_n) = 1$. Any data point for which $a_n = 0$ will not appear in the calculation of $y(\mathbf{x})$ and hence plays no role in making predictions for new data points. The remaining data points are called **support vectors**, and because they satisfy $t_n y(\mathbf{x}_n) = 1$.

Maximum Margin Classifiers

Main Algorithm:

Support vectors correspond to points that **lie on the maximum margin hyperplanes** in feature space. This property is central to the practical applicability of support vector machines. Once the model is trained, a significant proportion of the data points can be discarded and only the support vectors retained. Having solved the quadratic programming problem and found a value for $\mathbf{a}$, we can then determine the value of the **threshold** parameter b by noting that any support vector x_n satisfies $t_n y(x_n) = 1$.

$$t_n \left(\sum_{m \in \mathcal{S}} a_m t_m k(\mathbf{x}_n, \mathbf{x}_m) + b \right) = 1$$

Although we can solve this equation for b using an arbitrarily chosen support vector x_n , a numerically more stable solution is obtained by first multiplying through by t_n , making use of $t_n^2 = 1$, and then averaging these equations over all support vectors and solving for b to give:

$$b = \frac{1}{N_S} \sum_{n \in \mathcal{S}} \left(t_n - \sum_{m \in \mathcal{S}} a_m t_m k(\mathbf{x}_n, \mathbf{x}_m) \right)$$

Maximum Margin Classifiers

Main Algorithm:

For later comparison with alternative models, we can express the **maximum margin classifier** in terms of the minimization of an error function, with a simple quadratic regularizer, in the form of:

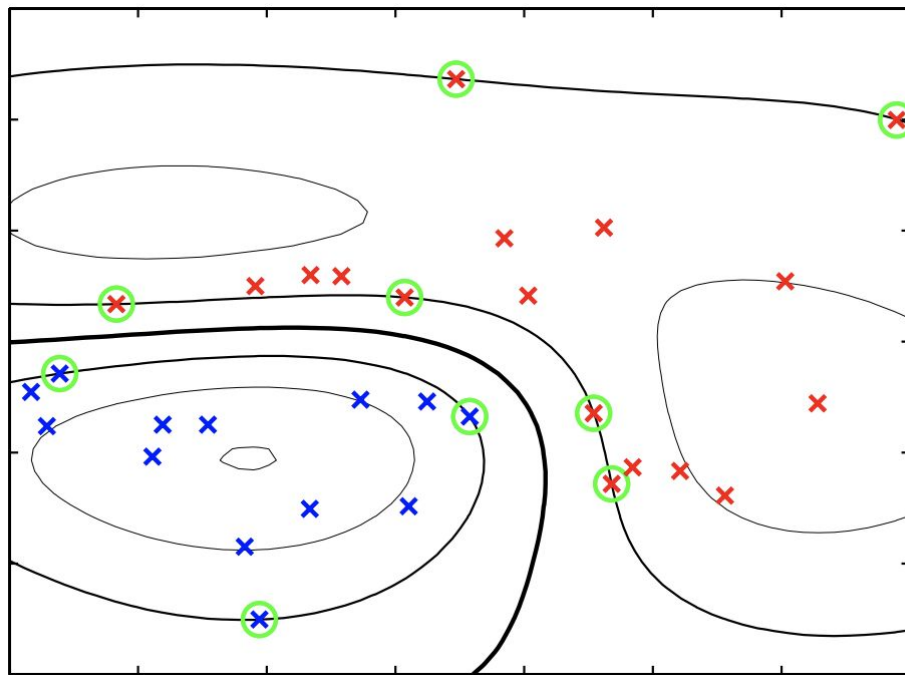
$$\sum_{n=1}^N E_{\infty}(y(\mathbf{x}_n)t_n - 1) + \lambda \|\mathbf{w}\|^2$$

where $E_{\infty}(\mathbf{z})$ is a function that is zero if $\mathbf{z} \geq 0$ and ∞ otherwise and ensures that the constraints are satisfied. Note that as long as the regularization parameter satisfies $\lambda > 0$, its precise value plays no role.

Maximum Margin Classifiers

Here we have an example of the classification resulting from training a support vector machine on a simple synthetic data set using a Gaussian kernel. Although the data set is not linearly separable in the two-dimensional data space $\mathbf{x}$, it is linearly separable in the nonlinear feature space defined implicitly by the nonlinear kernel function. Thus the training data points are perfectly separated in the original data space.

This example also provides a geometrical insight into the origin of sparsity in the SVM. The maximum margin hyperplane is defined by the location of the support vectors. Other data points can be moved around freely (so long as they remain outside the margin region) without changing the decision boundary, and so the solution will be independent of such data points.



Overlapping Class Distributions

Main Algorithm:

So far, we have assumed that the training data points are linearly separable in the feature space $\phi(\mathbf{x})$. The resulting support vector machine will give exact separation of the training data in the original input space $\mathbf{x}$, although the corresponding decision boundary will be nonlinear. In practice, however, the class-conditional distributions may overlap, in which case exact separation of the training data can lead to poor generalization. We therefore need a way to modify the support vector machine so as to allow some of the training points to be misclassified.

In the case of separable classes, we implicitly used an error function that gave **infinite error** if a data point was misclassified and **zero error** if it was classified correctly, and then optimized the model parameters to **maximize the margin**. We now modify this approach so that data points are allowed to be on the 'wrong side' of the margin boundary, but with a penalty that increases with the distance from that boundary. For the subsequent optimization problem, it is convenient to make this penalty a linear function of this distance.

Overlapping Class Distributions

Main Algorithm:

To do this, we introduce slack variables, $\xi_n \geq 0$ where $n = 1, \dots, N$, with one slack variable for each training data point. These are defined by $\xi_n = 0$ for data points that are on or inside the correct margin boundary and $\xi_n = |t_n - y(x_n)|$ for other points. Thus a data point that is on the decision boundary $y(x_n)=0$ will have $\xi_n = 1$, and points with $\xi_n > 1$ will be misclassified. The exact classification constraints are then replaced with:

$$t_n y(\mathbf{x}_n) \geq 1 - \xi_n, \quad n = 1, \dots, N$$

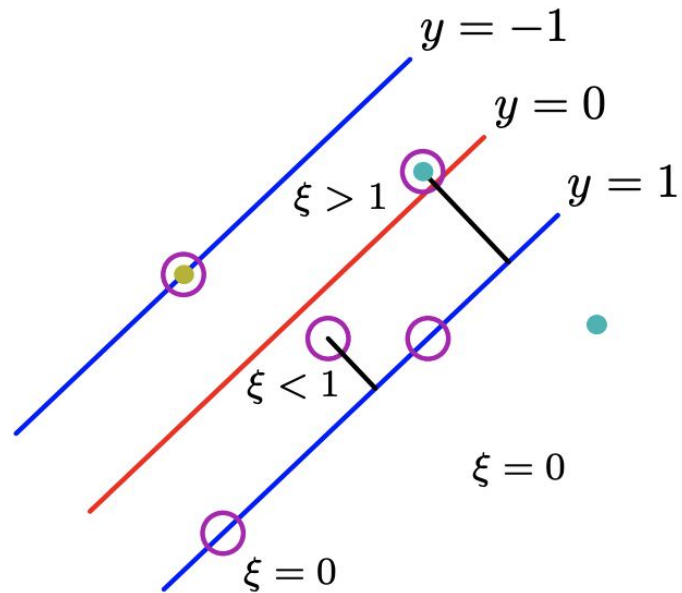
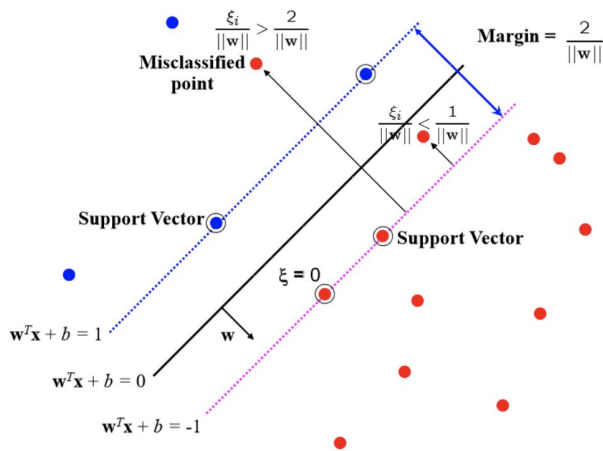
in which the slack variables are constrained to satisfy $\xi_n \geq 0$.

Data points for which

- $\xi_n = 0$ are correctly classified and are either on the margin or on the correct side of the margin.
- Points for which $0 < \xi_n \leq 1$ lie inside the margin, but on the correct side of the decision boundary.
- When $\xi_n > 1$ lie on the wrong side of the decision boundary and are misclassified

Overlapping Class Distributions

This is sometimes described as relaxing the hard margin constraint to give a soft margin and allows some of the training set data points to be misclassified. Note that while slack variables allow for overlapping class distributions, this framework is still sensitive to outliers because the penalty for misclassification increases linearly with ξ .



Overlapping Class Distributions

Main Algorithm:

Our goal is now to maximize the margin while softly penalizing points that lie on the wrong side of the margin boundary. We therefore minimize:

$$C \sum_{n=1}^N \xi_n + \frac{1}{2} \|\mathbf{w}\|^2$$

where the parameter $C > 0$ controls the trade-off between the slack variable penalty and the margin. Because any point that is misclassified has $\xi_n > 1$, it follows that $\sum_n \xi_n$ is an upper bound on the number of misclassified points. The parameter C is therefore analogous to (the inverse of) a regularization coefficient because it controls the trade-off between minimizing training errors and controlling model complexity. In the limit $C \rightarrow \infty$, we will recover the earlier support vector machine for separable data.

Overlapping Class Distributions

Main Algorithm:

We now wish to minimize subject to the constraints together with $\xi_n \geq 0$. The corresponding Lagrangian is given by

$$L(\mathbf{w}, b, \mathbf{a}) = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{n=1}^N \xi_n - \sum_{n=1}^N a_n \{t_n y(\mathbf{x}_n) - 1 + \xi_n\} - \sum_{n=1}^N \mu_n \xi_n$$

where $\{a_n \geq 0\}$ and $\{\mu_n \geq 0\}$ are Lagrange multipliers. The corresponding set of KKT conditions are given by:

$$\begin{aligned} a_n &\geq 0 \\ t_n y(\mathbf{x}_n) - 1 + \xi_n &\geq 0 \\ a_n (t_n y(\mathbf{x}_n) - 1 + \xi_n) &= 0 \\ \mu_n &\geq 0 \\ \xi_n &\geq 0 \\ \mu_n \xi_n &= 0 \end{aligned}$$

Overlapping Class Distributions

Main Algorithm:

We now optimize out $\mathbf{w}$, b , and $\{\xi_n\}$ making use of the definition of $y(\mathbf{x})$ to give:

$$\frac{\partial L}{\partial \mathbf{w}} = 0 \Rightarrow \mathbf{w} = \sum_{n=1}^N a_n t_n \phi(\mathbf{x}_n)$$

$$\frac{\partial L}{\partial b} = 0 \Rightarrow \sum_{n=1}^N a_n t_n = 0$$

$$\frac{\partial L}{\partial \xi_n} = 0 \Rightarrow a_n = C - \mu_n.$$

Using these results to eliminate $\mathbf{w}$, b , and $\{\xi_n\}$ from the Lagrangian, we obtain the dual Lagrangian in the form:

$$\tilde{L}(\mathbf{a}) = \sum_{n=1}^N a_n - \frac{1}{2} \sum_{n=1}^N \sum_{m=1}^N a_n a_m t_n t_m k(\mathbf{x}_n, \mathbf{x}_m)$$

Overlapping Class Distributions

Main Algorithm:

The **dual form** is identical to the separable case, except that the constraints are somewhat different. To see what these constraints are, we note that $a_n \geq 0$ is required because these are Lagrange multipliers. Furthermore, together with $\mu_n \geq 0$ implies $a_n \leq C$. We therefore have to minimize the Lagrangian with respect to the dual variables $\{a_n\}$ subject to:

$$0 \leq a_n \leq C \quad \sum_{n=1} a_n t_n = 0$$

for $n = 1, \dots, N$, where the first equation is known as **box constraints**.

We can now interpret the resulting solution. As before, a subset of the data points may have $a_n = 0$, in which case they do not contribute to the predictive model. The remaining data points constitute the **support vectors**. These have $a_n > 0$ and hence:

$$t_n y(\mathbf{x}_n) = 1 - \xi_n$$

Overlapping Class Distributions

Main Algorithm:

If $a_n < C$, then it implies that $\mu_n > 0$, which requires $\xi_n = 0$ and hence such points lie on the margin. Points with $a_n = C$ can lie inside the margin and can either be correctly classified if $\xi_n \leq 1$ or misclassified if $\xi_n > 1$.

To determine the parameter b , we note that those support vectors for which $0 < a_n < C$ have $\xi_n = 0$ so that $t_n y(x_n) = 1$ and hence will satisfy:

$$t_n \left(\sum_{m \in \mathcal{S}} a_m t_m k(\mathbf{x}_n, \mathbf{x}_m) + b \right) = 1.$$

Again, a numerically stable solution is obtained by averaging and given that $\mathcal{M}$ denotes the set of indices of data points having $0 < a_n < C$.

$$b = \frac{1}{N_{\mathcal{M}}} \sum_{n \in \mathcal{M}} \left(t_n - \sum_{m \in \mathcal{S}} a_m t_m k(\mathbf{x}_n, \mathbf{x}_m) \right)$$

Multiclass SVMs

Main Algorithm:

The support vector machine is fundamentally a two-class classifier. In practice, however, we often have to tackle problems involving $K > 2$ classes. Various methods have therefore been proposed for combining multiple two-class SVMs in order to build a multiclass classifier.

One commonly used approach is to construct K separate SVMs, in which the k th model $y_k(x)$ is trained using the data from class C_k as the positive examples and the data from the remaining $K - 1$ classes as the negative examples. This is known as the one-versus-the-rest approach. However using the decisions of the individual classifiers can lead to inconsistent results in which an input is assigned to multiple classes simultaneously. This problem is sometimes addressed by making predictions for new inputs x using:

$$y(\mathbf{x}) = \max_k y_k(\mathbf{x}).$$

Unfortunately, different classifiers were trained on different tasks, and there is no guarantee that the real valued quantities $y_k(x)$ for different classifiers will have appropriate scales.

Multiclass SVMs

Main Algorithm:

Another problem with the one-versus-the-rest approach is that the training sets are imbalanced. For instance, if we have ten classes each with equal numbers of training data points, then the individual classifiers are trained on data sets comprising 90% negative examples and only 10% positive examples, and the symmetry of the original problem is lost. A variant of the one-versus-the-rest scheme modifies the target values so that the positive class has target $+1$ and the negative class has target $-1/(K - 1)$.

Another approach is to train $K(K - 1)/2$ different 2-class SVMs on all possible pairs of classes, and then to classify test points according to which class has the highest number of 'votes', an approach that is sometimes called one-versus-one.

There are also single-class support vector machines, which solve an unsupervised learning problem related to probability density estimation. Instead of modelling the density of data, however, these methods aim to find a smooth boundary enclosing a region of high density.

SVM for Regression

Problem Formulation:

We now extend support vector machines to regression problems while at the same time preserving the property of sparseness. In simple linear regression, we minimize a regularized error function given by

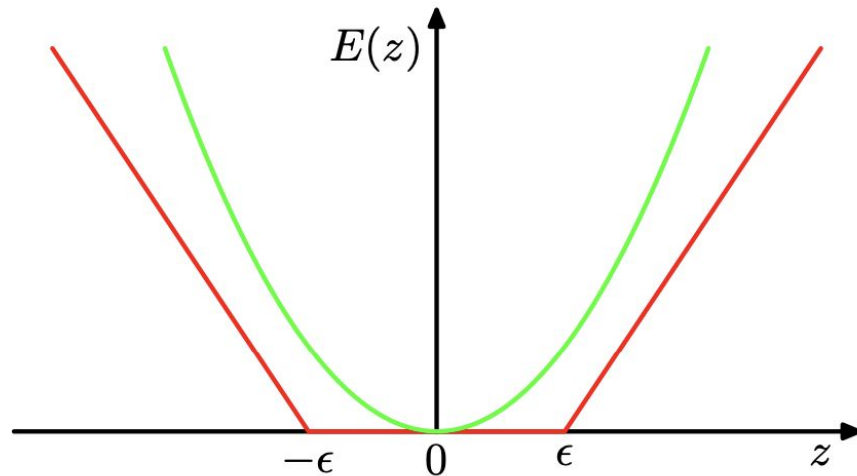
$$\frac{1}{2} \sum_{n=1}^N \{y_n - t_n\}^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

To obtain sparse solutions, the quadratic error function is replaced by an ϵ -insensitive error function, which gives zero error if the absolute difference between the prediction $y(\mathbf{x})$ and the target t is less than ϵ , where $\epsilon > 0$. A simple example of an ϵ -insensitive error function, having a linear cost associated with errors outside the insensitive region, is given by:

$$E_{\epsilon}(y(\mathbf{x}) - t) = \begin{cases} 0, & \text{if } |y(\mathbf{x}) - t| < \epsilon; \\ |y(\mathbf{x}) - t| - \epsilon, & \text{otherwise} \end{cases}$$

SVM for Regression

Plot of an ϵ -insensitive error function (in red) in which the error increases linearly with distance beyond the insensitive region. Also shown for comparison is the quadratic error function (in green).



SVM for Regression

Main Algorithm:

By convention we can put the (inverse) regularization parameter, denoted C , in front of the error term. We therefore minimize a regularized error function given by:

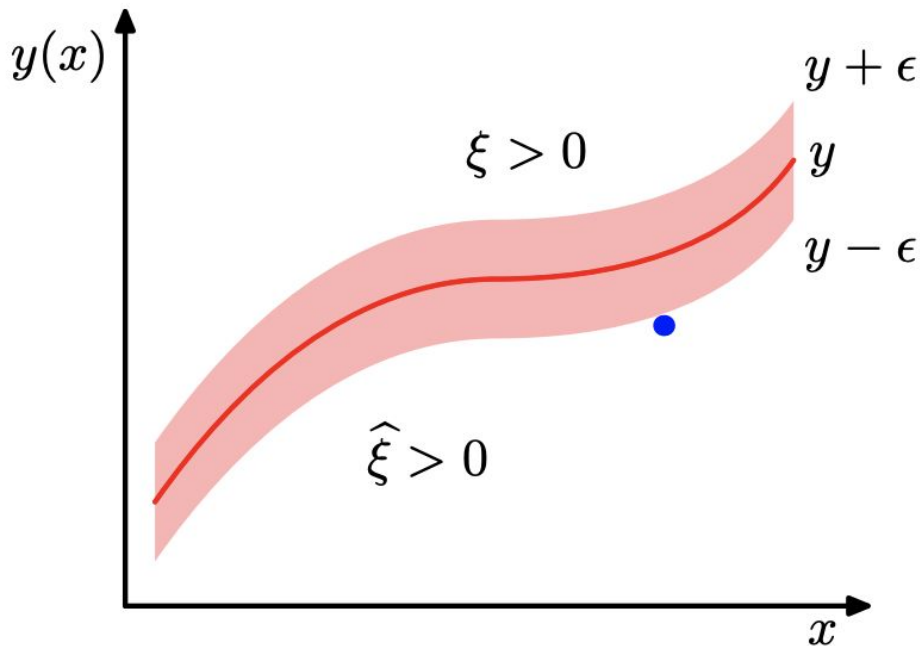
$$C \sum_{n=1}^N E_{\epsilon}(y(\mathbf{x}_n) - t_n) + \frac{1}{2} \|\mathbf{w}\|^2$$

As before, we can re-express the optimization problem by introducing slack variables. For each data point x_n , we now need two slack variables $\xi_n \geq 0$ and $\xi_n^{\wedge} \geq 0$, where $\xi_n > 0$ corresponds to a point for which $t_n > y(x_n) + \epsilon$, and $\xi_n^{\wedge} > 0$ corresponds to a point for which $t_n < y(x_n) - \epsilon$. The condition for a target point to lie inside the ϵ -tube is that $y_n - \epsilon \leq t_n \leq y_n + \epsilon$, where $y_n = y(x_n)$. Introducing the slack variables allows points to lie outside the tube provided the slack variables are nonzero, and the corresponding conditions are

$$\begin{aligned} t_n &\leq y(\mathbf{x}_n) + \epsilon + \xi_n \\ t_n &\geq y(\mathbf{x}_n) - \epsilon - \xi_n^{\wedge}. \end{aligned}$$

SVM for Regression

Illustration of SVM regression, showing the regression curve together with the ϵ -insensitive 'tube'. Also shown are examples of the slack variables ξ and $\hat{\xi}$. Points above the ϵ -tube have $\xi > 0$ and $\hat{\xi} = 0$, points below the ϵ -tube have $\xi = 0$ and $\hat{\xi} > 0$, and points inside the ϵ -tube have $\xi = \hat{\xi} = 0$.



SVM for Regression

Main Algorithm:

The error function for support vector regression can then be written as:

$$C \sum_{n=1}^N (\xi_n + \hat{\xi}_n) + \frac{1}{2} \|\mathbf{w}\|^2$$

which must be minimized subject to the constraints $\xi_n \geq 0$ and $\hat{\xi}_n \geq 0$. This can be achieved by introducing Lagrange multipliers $a_n \geq 0$, $\hat{a}_n \geq 0$, $\mu_n \geq 0$, and $\hat{\mu}_n \geq 0$ and optimizing the Lagrangian

$$\begin{aligned} L = & C \sum_{n=1}^N (\xi_n + \hat{\xi}_n) + \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{n=1}^N (\mu_n \xi_n + \hat{\mu}_n \hat{\xi}_n) \\ & - \sum_{n=1}^N a_n (\epsilon + \xi_n + y_n - t_n) - \sum_{n=1}^N \hat{a}_n (\epsilon + \hat{\xi}_n - y_n + t_n). \end{aligned}$$

SVM for Regression

Main Algorithm:

We now substitute for $y(\mathbf{x})$ and then set the derivatives of the Lagrangian with respect to $\mathbf{w}$, b , ξ_n , and $\hat{\xi}_n$ to zero, giving:

$$\frac{\partial L}{\partial \mathbf{w}} = 0 \quad \Rightarrow \quad \mathbf{w} = \sum_{n=1}^N (a_n - \hat{a}_n) \phi(\mathbf{x}_n)$$

$$\frac{\partial L}{\partial b} = 0 \quad \Rightarrow \quad \sum_{n=1}^N (a_n - \hat{a}_n) = 0$$

$$\frac{\partial L}{\partial \xi_n} = 0 \quad \Rightarrow \quad a_n + \mu_n = C$$

$$\frac{\partial L}{\partial \hat{\xi}_n} = 0 \quad \Rightarrow \quad \hat{a}_n + \hat{\mu}_n = C.$$

SVM for Regression

Main Algorithm:

Using these results to eliminate the corresponding variables from the Lagrangian, we see that the dual problem involves maximizing:

$$\begin{aligned}\tilde{L}(\mathbf{a}, \hat{\mathbf{a}}) = & -\frac{1}{2} \sum_{n=1}^N \sum_{m=1}^N (a_n - \hat{a}_n)(a_m - \hat{a}_m) k(\mathbf{x}_n, \mathbf{x}_m) \\ & -\epsilon \sum_{n=1}^N (a_n + \hat{a}_n) + \sum_{n=1}^N (a_n - \hat{a}_n) t_n\end{aligned}$$

with respect to $\{a_n\}$ and $\{\hat{a}_n\}$, where we have introduced the kernel $k(\mathbf{x}, \mathbf{x}') = \varphi(\mathbf{x})^\top \varphi(\mathbf{x}')$. Again, this is a constrained maximization, and to find the constraints we note that $a_n \geq 0$ and $\hat{a}_n \geq 0$ are both required because these are Lagrange multipliers. Also $\mu_n \geq 0$ and $\mu_n^\wedge \geq 0$ require $a_n \leq C$ and $\hat{a}_n \leq C$, and so again we have the box constraints

$$\begin{aligned}0 &\leq a_n \leq C \\ 0 &\leq \hat{a}_n \leq C\end{aligned}$$

SVM for Regression

Main Algorithm:

Substituting, we see that predictions for new inputs can be made using:

$$y(\mathbf{x}) = \sum_{n=1}^N (a_n - \hat{a}_n) k(\mathbf{x}, \mathbf{x}_n) + b$$

which is again expressed in terms of the kernel function. The corresponding Karush-Kuhn-Tucker (KKT) conditions, which state that at the solution the product of the dual variables and the constraints must vanish, are given by

$$\begin{aligned} a_n(\epsilon + \xi_n + y_n - t_n) &= 0 \\ \hat{a}_n(\epsilon + \hat{\xi}_n - y_n + t_n) &= 0 \\ (C - a_n)\xi_n &= 0 \\ (C - \hat{a}_n)\hat{\xi}_n &= 0. \end{aligned}$$

SVM for Regression

Main Algorithm:

From these we can obtain several useful results. First of all, we note that a coefficient a_n can only be nonzero if $\epsilon + \xi_n + y_n - t_n = 0$, which implies that the data point either lies on the upper boundary of the ϵ -tube ($\xi_n = 0$) or lies above the upper boundary ($\xi_n > 0$). Similarly, a nonzero value for $\hat{a}_n$ implies $\epsilon + \hat{\xi}_n - y_n + t_n = 0$, and such points must lie either on or below the lower boundary of the ϵ -tube. Furthermore, the two constraints $\epsilon + \xi_n + y_n - t_n = 0$ and $\epsilon + \hat{\xi}_n - y_n + t_n = 0$ are incompatible, as is easily seen by adding them together and noting that ξ_n and $\hat{\xi}_n$ are nonnegative while ϵ is strictly positive, and so for every data point $\mathbf{x}_n$, either a_n or $\hat{a}_n$ (or both) must be zero.

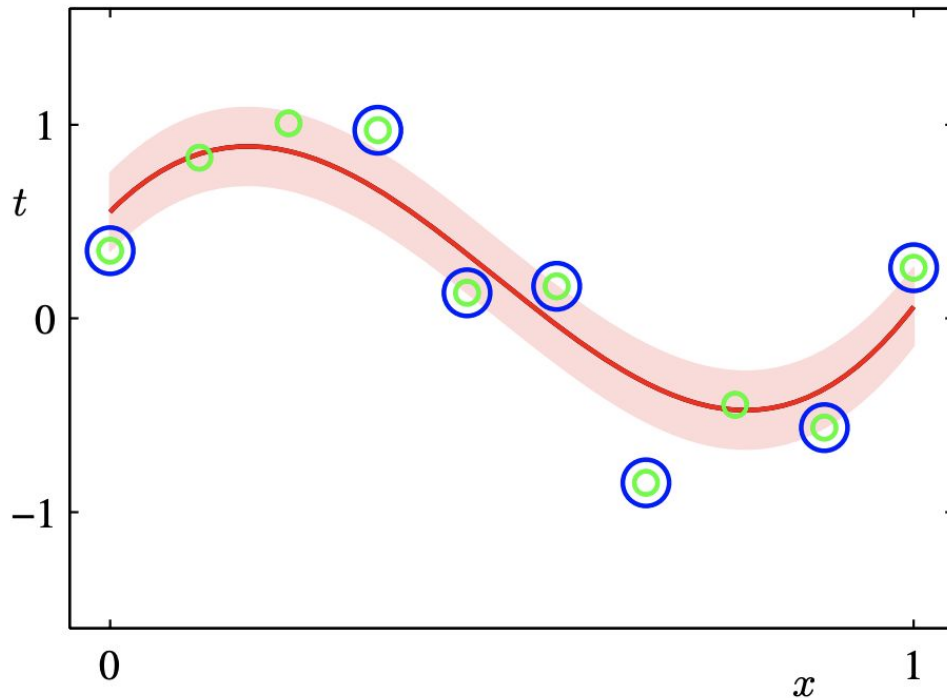
The support vectors are those data points for which either $a_n \neq 0$ or $\hat{a}_n \neq 0$. These are points that lie on the boundary of the ϵ -tube or outside the tube. All points within the tube have $a_n = \hat{a}_n = 0$.

The parameter b can be found by considering a data point for which $0 < a_n < C$, which must have $\xi_n = 0$, and must therefore satisfy $\epsilon + y_n - t_n = 0$.

$$b = t_n - \epsilon - \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n) = t_n - \epsilon - \sum_{m=1}^N (a_m - \hat{a}_m) k(\mathbf{x}_n, \mathbf{x}_m)$$

SVM for Regression

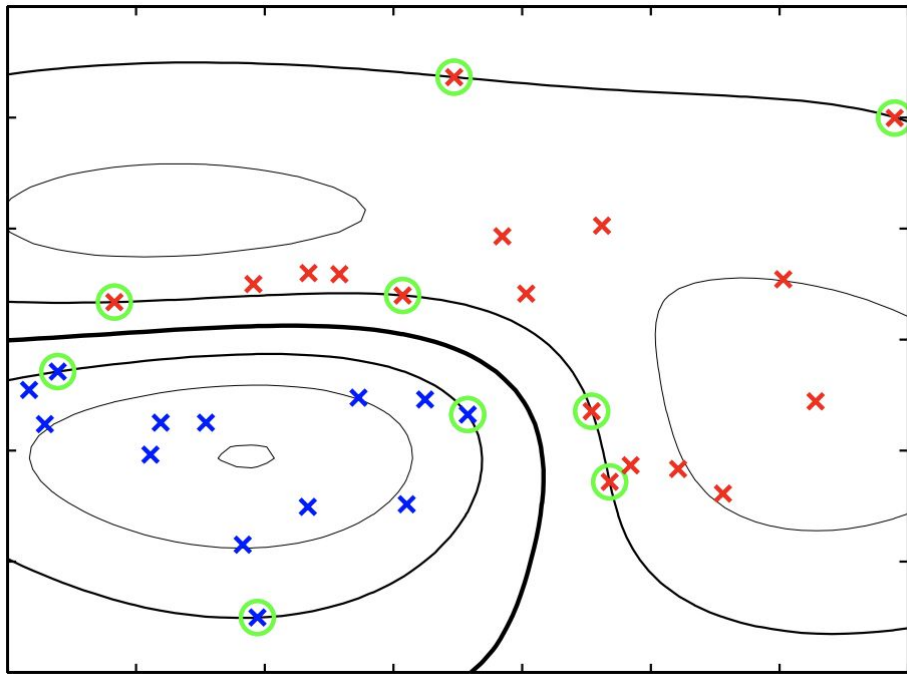
Illustration of the ν -SVM for regression applied to the sinusoidal synthetic data set using Gaussian kernels. The predicted regression curve is shown by the red line, and the ϵ -insensitive tube corresponds to the shaded region. Also, the data points are shown in green, and those with support vectors are indicated by blue circles.



Non-linear SVM Example

Here we have an example of the classification resulting from training a support vector machine on a simple synthetic data set using a Gaussian kernel. Although the data set is not linearly separable in the two-dimensional data space $\mathbf{x}$, it is linearly separable in the nonlinear feature space defined implicitly by the nonlinear kernel function. Thus the training data points are perfectly separated in the original data space.

This example also provides a geometrical insight into the origin of sparsity in the SVM. The maximum margin hyperplane is defined by the location of the support vectors. Other data points can be moved around freely (so long as they remain outside the margin region) without changing the decision boundary, and so the solution will be independent of such data points.



Kernel Trick

Why mapping features is a terrible idea

- **The Setup:** From our Dual Formulation, we know the SVM only cares about the dot product of data points:

$$\sum \alpha_i \alpha_j y_i y_j (x_i^T x_j).$$

- **The Naive Approach:** To create non-linear boundaries, we map our data to a higher dimensional space $x \rightarrow \varphi(x)$. We then replace the dot product with $\varphi(x_i)^T \varphi(x_j)$.
- **The Nightmare:** Suppose we have a 100-pixel image (100 dimensions) and we want to map it to capture all 5th-degree polynomial interactions between pixels. The feature vector $\varphi(x)$ would have over **75 million dimensions**.
- **The Definition:** A "Kernel" is simply a function $K(x, z)$ that computes the dot product of $\varphi(x)$ and $\varphi(z)$ *without ever actually calculating $\varphi(x)$ or $\varphi(z)$* .

Kernel Trick

The Kernel Trick in Action

Let's prove mathematically that this computational shortcut exists.

- **The Vectors:** Let $x = [x_1, x_2]^T$ and $z = [z_1, z_2]^T$.
- **The Feature Map:** Let's define a specific mapping φ that takes our 2D vectors into 3D:

$$\varphi(x) = [x_1^2, x_2^2, \sqrt{2}x_1x_2]^T$$

- **The Long Way (Feature Space Dot Product):**

$$\varphi(x)^T \varphi(z) = (x_1^2)(z_1^2) + (x_2^2)(z_2^2) + (\sqrt{2}x_1x_2)(\sqrt{2}z_1z_2)$$

$$\varphi(x)^T \varphi(z) = x_1^2z_1^2 + x_2^2z_2^2 + 2x_1x_2z_1z_2$$

- **The Factorization:** Notice that the result above is a perfect square!

$$x_1^2z_1^2 + x_2^2z_2^2 + 2x_1x_2z_1z_2 = (x_1z_1 + x_2z_2)^2$$

- **The Trick:** The term inside the parenthesis is just the original 2D dot product!

$$\varphi(x)^T \varphi(z) = (x^T z)^2 = K(x, z)$$

- **The Takeaway:** To find the dot product in 3D, we just did a dot product in 2D and squared it. $O(D^2)$ complexity is reduced to $O(D)$. As dimensions scale, this trick saves billions of calculations.

Kernel Trick

Mercer's Theorem

- **The Question:** If we don't need to compute $\phi(x)$, can I just invent any function, say $K(x, z) = \sin(x^T z)$, and use it as an SVM kernel?
- **The Answer:** No. A function K is only a valid Kernel if it corresponds to a dot product in *some* legitimate feature space.
- **Mercer's Theorem:** A function $K(x, z)$ is a valid kernel if and only if, for any finite dataset $\{x_1, \dots, x_N\}$, the resulting $N \times N$ Kernel Matrix (or Gram Matrix) K is **Positive Semi-Definite (PSD)**.
 - *Recall from Linear Algebra:* A matrix is PSD if its eigenvalues are non-negative, meaning $v^T K v \geq 0$ for any non-zero vector v .
- **The Intuition:** The dot product measures the cosine of the angle between vectors (a measure of distance/similarity). If the matrix isn't PSD, it means the implied feature space contains "negative lengths" or imaginary geometries. Mercer's theorem guarantees our math stays grounded in a real Hilbert space.

Kernel Trick

RBF Kernel

Let's look at the most popular kernel: the Gaussian Radial Basis Function (RBF).

$$K(x, z) = e^{-\gamma ||x - z||^2}$$

- **The Claim:** The feature space $\varphi(x)$ implied by the RBF kernel has an infinite number of dimensions. How is this possible?
- **The Proof (via Maclaurin Series):** Let's simplify and look at a 1D scalar kernel $K(x, z) = e^{xz}$.
- Expand e^{xz} using its infinite Taylor series:

$$e^{xz} = 1 + xz + (xz)^2 / 2! + (xz)^3 / 3! + \dots + (xz)^\infty / \infty!$$

- **The Breakdown:** Each term in this infinite sum is a polynomial kernel!
$$e^{xz} = (1)(1) + (x)(z) + (x^2 / \sqrt{2!}) (z^2 / \sqrt{2!}) + (x^3 / \sqrt{3!}) (z^3 / \sqrt{3!}) + \dots$$
- **The Reality:** By using the e^x function, the RBF kernel is computing the dot product for a feature vector $\varphi(x)$ that contains every possible polynomial degree of x , stretching out to infinity.
- **The true magic of the Kernel Trick:** We are finding an optimal hyperplane in an infinite-dimensional space, and it only costs us one $O(1)$ exponential operation per pair of points.

Kernel Trick

The Algebra of Kernels

Why Mercer's Theorem matters in practice: Because Mercer guarantees PSD matrices, we can use the properties of PSD matrices to build new, complex kernels out of simple ones, *without ever needing to know what their $\varphi(x)$ looks like*.

If K_1 and K_2 are valid kernels, then:

1. **Scaling:** cK_1 is valid ($c > 0$)
2. **Addition:** $K_1 + K_2$ is valid
3. **Multiplication:** $K_1 \times K_2$ is valid

Real-World Power: Data isn't always numerical vectors. It can be DNA strings, text documents, or graphs.

- If you can write a function that measures the similarity between two DNA strings, and you can prove that function yields a PSD matrix, **you have created a String Kernel**. You can now feed raw text directly into an SVM.